

Chapter 4

Applied Statistics and Exploratory Analysis

Statistical Measure

A statistical measure is a numerical value that summarizes, describes, or explains characteristics of data.

In data science, statistical measures help to:

- Understand data distribution
- Identify relationships between variables
- Detect patterns and anomalies
- Support decision-making and modeling

Covariance

Covariance measures the **direction of the relationship** between two variables.

- If covariance $> 0 \rightarrow$ variables increase together
- If covariance $< 0 \rightarrow$ one increases, other decreases
- If covariance $= 0 \rightarrow$ no linear relationship

Formula:

$$\text{Cov}(X, Y) = \frac{\sum (X_i - \bar{X})(Y_i - \bar{Y})}{n}$$

Covariance

```
import numpy as np

x = np.array([10, 20, 30, 40, 50])
y = np.array([15, 25, 35, 45, 60])

covariance = np.cov(x, y)[0, 1]
print("Covariance:", covariance)
```

Correlation

Correlation measures both strength and direction of the linear relationship between two variables.

- Range: -1 to +1
- +1 → perfect positive correlation
- -1 → perfect negative correlation
- 0 → no linear correlation

```
correlation = np.corrcoef(x, y)[0, 1]  
print("Correlation:", correlation)
```

Formula:

$$\text{Correlation} = \frac{\text{Cov}(X, Y)}{\sigma_X \sigma_Y}$$

Skewness

Skewness measures the **asymmetry** of a data distribution.

- **Skewness = 0** → symmetric distribution
- **Positive skew** → long tail on the right
- **Negative skew** → long tail on the left

Interpretation:

- **Mean > Median** → Positive skew
- **Mean < Median** → Negative skew

```
from scipy.stats import skew

data = [10, 20, 25, 30, 100]

skewness = skew(data)
print("Skewness:", skewness)
```

Kurtosis

Kurtosis is a statistical measure of the "tailedness" of a probability distribution, indicating the frequency of extreme outliers. It describes the shape of tails relative to a normal distribution, with three main types: Leptokurtic (fat tails, high risk), Mesokurtic (normal), and Platykurtic (thin tails).

```
from scipy.stats import kurtosis

normal_data = [10, 12, 14, 16, 18]
outlier_data = [10, 12, 14, 16, 100]

print("Normal-like kurtosis:", kurtosis(normal_data))
print("Outlier-heavy kurtosis:", kurtosis(outlier_data))
```

Probability review , sampling and hypothesis testing

Review from pdf

Simple Linear regression

- Simple linear regression models the relationship between two variables by fitting a linear equation to the observed data.
- The equation of the line is typically represented as:

$$Y = \beta_0 + \beta_1 X + \epsilon$$

Where:

- Y is the dependent variable (what you're predicting),
- X is the independent variable (the predictor),
- β_0 is the intercept (constant),
- β_1 is the slope (the change in Y for a unit change in X),
- ϵ is the error term (the difference between the predicted and actual values).

Multiple Linear Regression

- Multiple Linear Regression (MLR) is an extension of simple linear regression to include multiple independent variables (predictors). The general form is:

$$Y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \dots + \beta_p X_p + \epsilon$$

Where:

- Y : Dependent variable (target)
- β_0 : Intercept
- $\beta_1, \beta_2, \dots, \beta_p$: Regression coefficients
- $X_1, X_2, \dots, X_p$: Independent variables (predictors)
- ϵ : Error term

Regression using statsmodels

```
import statsmodels.api as sm
import numpy as np
```

Data

```
X = np.array([1, 2, 3, 4, 5])
Y = np.array([2, 4, 5, 4, 6])
```

Add constant for intercept

```
X = sm.add_constant(X)
```

Fit model

```
model = sm.OLS(Y, X)
results = model.fit()
```

Summary

```
print(results.summary())
```

```
X = np.array([
    [1, 2],
    [2, 3],
    [3, 4],
    [4, 5],
    [5, 6]
])
```

```
Y = np.array([5, 7, 9, 11, 13])
```

```
X = sm.add_constant(X)
```

```
model = sm.OLS(Y, X)
results = model.fit()
```

```
print(results.summary())
```

Trend Analysis

Trend analysis studies the long-term movement of a variable over time.

Time itself becomes the independent variable.

$$Y_t = \beta_0 + \beta_1 t + \varepsilon_t$$

```
data["time_sq"] = data["time"] ** 2

X = sm.add_constant(data[["time", "time_sq"]])
Y = data["sales"]

model = sm.OLS(Y, X).fit()
print(model.summary())
```

What is EDA (Exploratory Data Analysis)

Exploratory Data Analysis (EDA) is the process of analyzing and summarizing a dataset to understand its main characteristics, patterns, and relationships before applying any modeling or prediction.

What is EDA (Exploratory Data Analysis)

1. Purpose:

- Discover patterns, trends, and anomalies in data
- Check assumptions required for statistical models
- Prepare data for further analysis or machine learning

2. Nature:

- **Visual and statistical**
- Often iterative: you explore, visualize, clean, and repeat

3. Components of EDA:

- **Descriptive analysis:** Mean, median, mode, variance, distribution
- **Visual analysis:** Histograms, boxplots, scatterplots, correlation heatmaps
- **Inferential analysis:** Correlation, hypothesis testing, confidence intervals

Descriptive vs Inferential Methods

Descriptive methods are statistical techniques used to summarize, organize, and describe the main features of a dataset.

They describe what the data shows, without making conclusions beyond the given data.

What Descriptive Methods Do ?

- Summarize data
- Identify patterns and trends
- Detect outliers
- Understand data distribution

No generalization to population

Common Descriptive Methods

(a) Numerical Measures

- Mean
- Median
- Mode
- Range
- Variance
- Standard Deviation
- Skewness
- Kurtosis

(b) Graphical Methods

- Histogram
- Box plot
- Bar chart
- Pie chart
- Scatter plot

Inferential Methods

Inferential methods use sample data to **make predictions, conclusions, or decisions about a population.**

They account for **uncertainty and probability.**

What Inferential Methods Do ?

Generalize sample results to population

Test assumptions

Estimate population parameters

Support decision-making

Common Inferential Methods.

- Sampling techniques
- Confidence intervals
- Hypothesis testing
- t-test
- z-test
- Chi-square test
- ANOVA
- Regression analysis

When to use ?

Situation	Method
Summarizing dataset	Descriptive
Checking distribution	Descriptive
Comparing populations	Inferential
Making predictions	Inferential
Decision under uncertainty	Inferential

Automation of EDA Workflow

1. Data Loading

- CSV, Excel, SQL, JSON

2. Initial Inspection

- Dataset shape, types, missing values, duplicates

3. Data Cleaning

- Handle missing values
- Correct data types
- Remove duplicates/outliers

4. Descriptive Analysis

- Mean, median, mode
- Variance, std deviation, skewness, kurtosis
- Frequency counts for categorical variables

5. Inferential Analysis

- Correlation
- Hypothesis testing (t-test, chi-square)

6. Visualizations

- Histograms, boxplots, bar plots, scatter plots
- Correlation heatmap

7. Automated Reporting

- Generate HTML/interactive report

8. Export Cleaned Data

- For modeling or further analysis

Code Example Using Python

Library	Purpose
pandas	Data handling, descriptive stats
numpy	Numeric operations
matplotlib / seaborn	Visualization
scipy / statsmodels	Inferential statistics
pandas-profiling	Automated HTML EDA report
sweetviz	Visual report, comparison
warnings	Suppress unnecessary warnings

Load module and load data

```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
from scipy import stats
from pandas_profiling import ProfileReport
import sweetviz as sv
import warnings
warnings.filterwarnings("ignore")
```

```
def load_data(file_path):
    if file_path.endswith('.csv'):
        df = pd.read_csv(file_path)
    elif file_path.endswith('.xlsx'):
        df = pd.read_excel(file_path)
    else:
        raise ValueError("Unsupported file format")
    return df
```


Data inspection and data cleaning

```
def inspect_data(df):  
    print("==== Dataset Info =====")  
    print(df.info())  
    print("\n==== Dataset Head =====")  
    print(df.head())  
    print("\n==== Missing Values =====")  
    print(df.isnull().sum())  
    print("\n==== Duplicate Rows =====")  
    print(df.duplicated().sum())
```

```
def clean_data(df):  
    # Remove duplicates  
    df = df.drop_duplicates()  
  
    # Handle missing numeric values  
    for col in df.select_dtypes(include=np.number):  
        df[col] = df[col].fillna(df[col].median())  
  
    # Handle missing categorical values  
    for col in df.select_dtypes(include='object'):  
        df[col] = df[col].fillna(df[col].mode()[0])  
  
    return df
```

Descriptive statistics

```
def descriptive_stats(df):  
    print("\n==== Descriptive Statistics =====")  
    print(df.describe(include='all'))  
  
    # Skewness & Kurtosis  
    numeric_cols = df.select_dtypes(include=np.number).columns  
    print("\n==== Skewness & Kurtosis =====")  
    for col in numeric_cols:  
        print(f"{col} -> Skewness: {df[col].skew():.2f}, Kurtosis: {df[col].kurt():.2f}")
```

Inferential Statistics

```
def inferential_analysis(df):  
    print("\n==== Correlation Matrix =====")  
    corr = df.corr()  
    print(corr)  
  
    plt.figure(figsize=(8,6))  
    sns.heatmap(corr, annot=True, cmap='coolwarm')  
    plt.title("Correlation Heatmap")  
    plt.show()
```

Hypothesis testing (Optional)

```
def t_test_example(df, col1, col2):  
    if col1 in df.columns and col2 in df.columns:  
        t_stat, p_val = stats.ttest_ind(df[col1], df[col2], equal_var=False)  
        print(f"T-test between {col1} and {col2}: t={t_stat:.2f}, p={p_val:.4f}")  
    else:  
        print("Columns not found for t-test.")
```

Data Visualization

```
def visualize_data(df):  
    numeric_cols = df.select_dtypes(include=np.number).columns  
    categorical_cols = df.select_dtypes(include='object').columns  
  
    # Numeric plots  
    for col in numeric_cols:  
        plt.figure(figsize=(6,3))  
        sns.histplot(df[col], kde=True)  
        plt.title(f"Distribution of {col}")  
        plt.show()  
  
        plt.figure(figsize=(6,3))  
        sns.boxplot(x=df[col])  
        plt.title(f"Boxplot of {col}")  
        plt.show()
```

```
for col in categorical_cols:  
    plt.figure(figsize=(6,3))  
    sns.countplot(y=df[col])  
    plt.title(f"Countplot of {col}")  
    plt.show()
```

Report Generation

```
def generate_reports(df):  
    # pandas-profiling  
    profile = ProfileReport(df, title="EDA Report", explorative=True)  
    profile.to_file("eda_report.html")  
  
    # Sweetviz  
    report = sv.analyze(df)  
    report.show_html("sweetviz_report.html")  
  
    print("Reports generated: eda_report.html & sweetviz_report.html")
```

Save the cleaned data

```
def save_cleaned_data(df, file_name="cleaned_data.csv"):
    df.to_csv(file_name, index=False)
    print(f"Cleaned data saved as {file_name}")
```

Merge everything

```
def automate_eda(file_path):  
    df = load_data(file_path)  
    inspect_data(df)  
    df = clean_data(df)  
    descriptive_stats(df)  
    inferential_analysis(df)  
    visualize_data(df)  
    generate_reports(df)  
    save_cleaned_data(df)
```

```
file_path = "your_dataset.csv"  
automate_eda(file_path)
```